MediScribe AI — Master Build Prompt

Universal Scaffold Prompt for Claude / OpenAI Playground / Google AI Studio / Codex / Windsurf / Firebase

> How to use: Copy everything from the horizontal rule below into your target platform. Works as a single-shot scaffold prompt in any LLM-powered IDE or API playground. Pair with Firebase Console for infra setup.

MEDISCRIBE AI — FULL-STACK BUILD SPECIFICATION

You are a senior full-stack engineer and UX architect. Build the complete project scaffold for MediScribe AI — a clinician-first, RAG-powered clinical journaling and patient management platform. Read this specification completely before writing a single line of code.

---

1. PRODUCT VISION

MediScribe AI is a unified mobile + web workspace for doctors that consolidates:

- Handwritten notes, voice dictation, PDFs, scanned records
- Patient history and document management
- AI-powered search grounded exclusively in the clinician's own uploaded data
- Appointment calendar and session history

Core philosophy: The AI never hallucinates. Every answer is sourced from the doctor's own documents. If the information is not in the knowledge base, the system says so.

Target users: Medical professionals (primary), medical students (future phase)

Competitors to surpass: Apple Journal, Obsidian, Scrivener — surpass them in clinical utility, UI simplicity, and AI capability.

---

2. DIRECTORY STRUCTURE

Generate the following complete monorepo:

```
mediscribe-ai/
├── apps/
│   ├── mobile/                    # React Native (Expo) — iOS & Android
│   │   ├── src/
│   │   │   ├── components/
│   │   │   │   ├── shared/
│   │   │   │   │   ├── PersistentCalendar.tsx
│   │   │   │   │   ├── AISearchButton.tsx
│   │   │   │   │   ├── MiniArchive.tsx
│   │   │   │   │   └── SessionHistory.tsx
│   │   │   │   ├── journal/
│   │   │   │   │   ├── NoteEditor.tsx       # Rich text + Markdown
│   │   │   │   │   ├── ScribbleCanvas.tsx    # Stylus/ink-to-text
│   │   │   │   │   ├── VoiceRecorder.tsx     # STT capture
│   │   │   │   │   ├── DocumentViewer.tsx    # PDF/image render
│   │   │   │   │   ├── AskAIButton.tsx       # Document-scoped RAG
│   │   │   │   │   ├── SummaryStrip.tsx      # 1-line / 3-bullet / 1-para
│   │   │   │   │   └── ResearchArchive.tsx
│   │   │   │   └── patient/
│   │   │   │       ├── PatientCard.tsx
│   │   │   │       ├── PatientTimeline.tsx
│   │   │   │       ├── DocumentHistory.tsx
│   │   │   │       └── PatientSearch.tsx
│   │   │   ├── screens/
│   │   │   │   ├── HomeScreen.tsx           # Toggle + persistent layer
│   │   │   │   ├── JournalScreen.tsx
│   │   │   │   ├── PatientScreen.tsx
│   │   │   │   ├── DocumentDetailScreen.tsx
│   │   │   │   ├── ChatScreen.tsx           # Document-scoped AI chat
│   │   │   │   └── SettingsScreen.tsx
│   │   │   ├── navigation/
│   │   │   │   └── AppNavigator.tsx
│   │   │   ├── store/
│   │   │   │   ├── index.ts                 # Redux Toolkit store
│   │   │   │   ├── journalSlice.ts
│   │   │   │   ├── patientSlice.ts
│   │   │   │   └── uiSlice.ts
│   │   │   ├── hooks/
│   │   │   │   ├── useRAGQuery.ts
│   │   │   │   ├── useSTT.ts
│   │   │   │   └── useOCR.ts
│   │   │   ├── services/
│   │   │   │   ├── api.ts                   # Backend API client
│   │   │   │   ├── firebase.ts              # Firebase SDK config
│   │   │   │   └── pinecone.ts              # Vector DB helper
│   │   │   ├── utils/
```

```
│   │   │   │   ├── chunker.ts
│   │   │   │   └── formatter.ts
│   │   │   └── theme/
│   │   │       ├── colors.ts
│   │   │       ├── typography.ts
│   │   │       └── spacing.ts
│   │   ├── app.json
│   │   ├── package.json
│   │   └── tsconfig.json
│   │
│   └── web/                        # React (Vite) — desktop browser
│       ├── src/
│       │   ├── components/         # Mirror mobile components, adapted for 2-pane layout
│       │   ├── pages/
│       │   ├── store/
│       │   └── services/
│       ├── index.html
│       ├── vite.config.ts
│       └── package.json
│
├── functions/                      # Firebase Cloud Functions (Node.js + TypeScript)
│   ├── src/
│   │   ├── ingest/
│   │   │   ├── pdfIngest.ts        # PDF → OCR → chunk → embed → Pinecone
│   │   │   ├── audioIngest.ts      # Audio → Whisper → chunk → embed → Pinecone
│   │   │   ├── imageIngest.ts      # Image/handwriting → OCR → chunk → embed
│   │   │   └── noteIngest.ts       # Text note → chunk → embed → Pinecone
│   │   ├── rag/
│   │   │   ├── query.ts            # Core RAG query handler
│   │   │   ├── hybridSearch.ts     # Dense + sparse (BM25) retrieval
│   │   │   └── reranker.ts         # Cross-encoder re-ranking
│   │   ├── ai/
│   │   │   ├── llmRouter.ts        # OpenAI / Claude / Gemini routing
│   │   │   ├── summarizer.ts       # Note + doc summarization
│   │   │   ├── soapGenerator.ts    # SOAP note auto-generation from STT
│   │   │   └── entityExtractor.ts  # NLP: extract patient events
│   │   ├── api/
│   │   │   ├── routes.ts           # Express router
│   │   │   ├── authMiddleware.ts   # Firebase Auth token verification
│   │   │   └── rateLimiter.ts
│   │   └── index.ts                # Function exports
│   ├── package.json
│   └── tsconfig.json
│
├── infra/
│   ├── terraform/
│   │   ├── main.tf                 # GCP + Pinecone provisioning
```

```
│   │   ├── firebase.tf
│   │   └── variables.tf
│   └── .env.example                # All required environment variables
│
├── scripts/
│   ├── seed-pinecone.ts            # Seed vector DB with test data
│   └── test-rag.ts                 # RAG pipeline smoke test
│
├── .github/
│   └── workflows/
│       ├── ci.yml                  # Lint + test on PR
│       └── deploy.yml              # Deploy to Cloud Run + Firebase Hosting
│
├── package.json                    # Monorepo root (workspaces)
└── README.md
```

---

3. TECH STACK

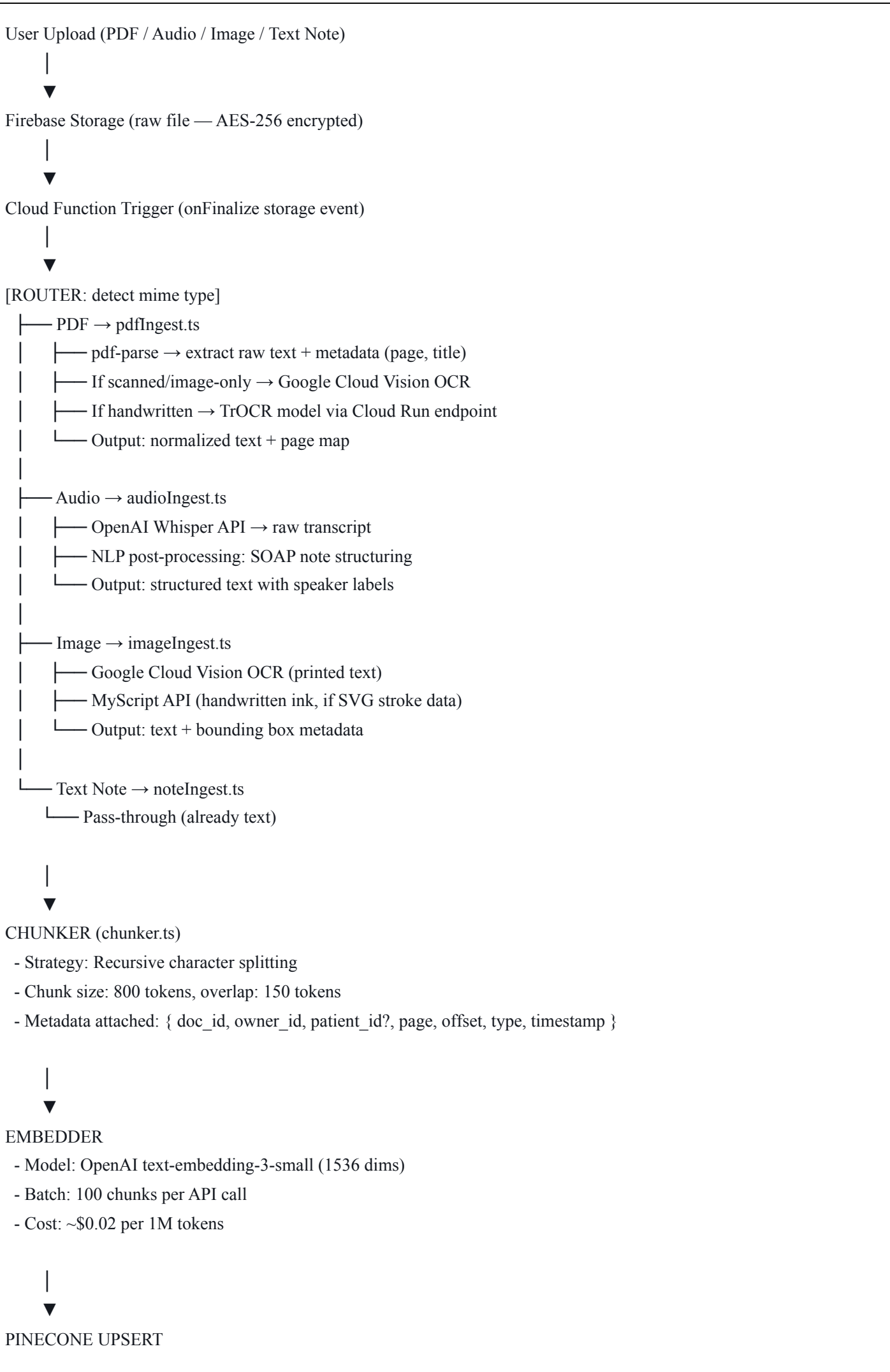
Layer	Technology	Reason
Mobile	React Native (Expo SDK 51)	Cross-platform iOS/Android, Expo Go for rapid testing
Web	React + Vite + TypeScript	Desktop 2-pane experience
State	Redux Toolkit + React Query	Global state + server sync
Offline	expo-sqlite + SQLite	Local persistence when offline
Backend	Firebase Cloud Functions (Node.js 20 + TypeScript)	Serverless, auto-scaling
Auth	Firebase Authentication (Email + MFA + Biometric)	HIPAA-ready, zero config
Database	Cloud Firestore	Real-time sync, offline support
File Storage	Firebase Storage	PDFs, audio, images — encrypted at rest
Vector DB	Pinecone (Serverless, us-east-1)	Managed RAG, 100K free vectors
Embeddings	OpenAI `text-embedding-3-small` (swap: Google `text-embedding-004`)	Best price/performance
LLM Primary	OpenAI GPT-4o	Complex reasoning, multimodal
LLM Secondary	Anthropic Claude 3.5 Sonnet	Safety-critical fallback

Layer	Technology	Reason
LLM Tertiary	Google Gemini 2.0 Flash	Fast/cheap for summaries
OCR	Google Cloud Vision API + TrOCR (handwriting)	95%+ accuracy on messy clinical notes
STT	OpenAI Whisper API (medical vocabulary)	WER < 5% on clinical speech
Handwriting	MyScript iInk SDK	Native stylus ink-to-text on iPad/Android
Charts	Recharts (web) + Victory Native (mobile)	Pie, bar, line for summaries
PDF Export	@react-pdf/renderer + Puppeteer (server-side)	Client + server PDF generation
CI/CD	GitHub Actions → Firebase Hosting + Cloud Run	Zero-downtime deploys
IaC	Terraform	Reproducible infra

4. FULL BACKEND ARCHITECTURE & ALGORITHMS

4.1 Ingestion Pipeline

Every uploaded artifact flows through this pipeline:


User Upload (PDF / Audio / Image / Text Note)
Firebase Storage (raw file — AES-256 encrypted)
Cloud Function Trigger (onFinalize storage event)
[ROUTER: detect mime type]
PDF → pdfIngest.ts
pdf-parse → extract raw text + metadata (page, title)
If scanned/image-only → Google Cloud Vision OCR
If handwritten → TrOCR model via Cloud Run endpoint
Output: normalized text + page map
Audio → audioIngest.ts
OpenAI Whisper API → raw transcript
NLP post-processing: SOAP note structuring
Output: structured text with speaker labels
Image → imageIngest.ts
Google Cloud Vision OCR (printed text)
MyScript API (handwritten ink, if SVG stroke data)
Output: text + bounding box metadata
Text Note → noteIngest.ts
Pass-through (already text)
CHUNKER (chunker.ts)
- Strategy: Recursive character splitting
- Chunk size: 800 tokens, overlap: 150 tokens
- Metadata attached: { doc_id, owner_id, patient_id?, page, offset, type, timestamp }
EMBEDDER
- Model: OpenAI text-embedding-3-small (1536 dims)
- Batch: 100 chunks per API call
- Cost: ~$0.02 per 1M tokens
PINECONE UPSERT

- Namespace: `user_{user_id}` (strict data isolation per doctor)
- Vector: [embedding float[1536]]
- Metadata: { chunk_id, doc_id, text_preview, page, type, patient_id? }

▼

FIRESTORE UPDATE
- Document record status: "indexed"
- Chunk count stored for display

4.2 RAG Query Pipeline

User Query (natural language) + optional { doc_scope, patient_scope }

▼

QUERY PREPROCESSOR (query.ts)
- If clinical query: restructure using PICOT framework
  (Population, Intervention, Comparison, Outcome, Time)
- Detect query type: factual | comparative | summarization

▼

HYBRID RETRIEVAL (hybridSearch.ts)

├── Dense Search: embed query → Pinecone similarity search
│ - top_k: 20, namespace: `user_{user_id}`
│ - Filter: { doc_id: scope_doc_id } if doc-scoped chat
│
└── Sparse Search: BM25 over Firestore text index
  - Exact keyword matching for drug names, ICD codes
  - top_k: 20

▼

RECIPROCAL RANK FUSION
- Merge dense + sparse results using RRF algorithm
- Score: 1/(rank_dense + 60) + 1/(rank_sparse + 60)
- Select top 8 chunks

▼

RERANKER (reranker.ts)
- Cross-encoder: Cohere Rerank API (or local BGE reranker)
- Re-score top 8 → select top 5 for LLM context

▼

```
PROMPT ASSEMBLY
  System: "You are MediScribe AI, a clinical assistant. Answer ONLY using
  the numbered sources below. If the answer is not contained in the
  sources, respond exactly: 'Insufficient information in your documents.'
  Always cite source numbers [1], [2], etc."

  Context: <source_1>...</source_1> <source_2>...</source_2> ... <source_5>...
  User: {original_query}

      |
      ▼
LLM ROUTER (llmRouter.ts)
  - Default: OpenAI GPT-4o
  - Fallback (if OpenAI down): Anthropic Claude
  - Summaries only: Gemini 2.0 Flash (3x cheaper)

      |
      ▼
SELF-REFLECTIVE CHECK
  - Parse LLM response for citations [1]..[5]
  - If no citation found: flag as potential hallucination
  - Re-query with "verify your answer against provided sources"

      |
      ▼
RESPONSE + PROVENANCE
  { answer: string, sources: [{doc_id, page, text_preview, doc_title}] }
  → returned to frontend → displayed with inline citation chips
```

4.3 Data Models (Firestore)

typescript

```
// users collection
interface User {
  user_id: string;          // Firebase Auth UID
  name: string;
  email: string;
  role: 'doctor' | 'admin' | 'student';
  org_id?: string;
  mfa_enabled: boolean;
  biometric_enabled: boolean;
  created_at: Timestamp;
}

// documents collection
interface MedDocument {
  doc_id: string;           // UUID
  owner_id: string;         // user_id
  patient_id?: string;      // null for journal docs
  type: 'pdf' | 'audio' | 'image' | 'note' | 'scribble';
  title: string;
  storage_path: string;     // Firebase Storage path
  ocr_text_path?: string;   // Storage path to extracted text
  status: 'uploading' | 'processing' | 'indexed' | 'error';
  chunk_count: number;
  upload_ts: Timestamp;
  tags: string[];
  is_archived: boolean;
}

// patients collection
interface Patient {
  patient_id: string;
  owner_id: string;
  name_hash: string;        // SHA-256 hash — PII minimization
  dob_encrypted: string;    // AES-256 encrypted
  gender?: string;
  identifiers: { type: string; value_encrypted: string }[];
  consent_flags: { research: boolean; sharing: boolean };
  created_at: Timestamp;
}

// sessions collection (audit trail)
interface Session {
  session_id: string;
  user_id: string;
  doc_id?: string;
  patient_id?: string;
```

```
  queries: { query: string; response_preview: string; ts: Timestamp }[];
  started_at: Timestamp;
  ended_at?: Timestamp;
}

// appointments collection
interface Appointment {
  appt_id: string;
  owner_id: string;
  patient_id?: string;
  title: string;
  type: 'surgery' | 'consultation' | 'followup' | 'other';
  start_ts: Timestamp;
  end_ts: Timestamp;
  location?: string;
  notes?: string;
  reminders: number[];      // minutes before: [30, 10]
  created_at: Timestamp;
}

// chunks collection (Firestore — mirrors Pinecone for metadata queries)
interface Chunk {
  chunk_id: string;
  doc_id: string;
  owner_id: string;
  text_preview: string;     // first 200 chars
  page?: number;
  offset: number;
  embedding_id: string;     // Pinecone vector ID
  created_at: Timestamp;
}
```

4.4 REST API Endpoints (Firebase Cloud Functions)

```
POST   /api/ingest              Upload file → trigger pipeline
                        Body: multipart/form-data { file, patient_id? }
                        Response: { job_id, doc_id, status }

POST   /api/notes               Create text/scribble note
                        Body: { title, content, patient_id?, tags[] }
                        Response: { doc_id }

POST   /api/query               RAG query
                        Body: { query, doc_scope?: doc_id, patient_scope?: patient_id }
                        Response: { answer, sources[], session_id }
```

```
GET    /api/documents                List user's documents
                                     Query: ?patient_id=&type=&limit=20&cursor=

GET    /api/documents/:doc_id        Get document details + chunk count

DELETE /api/documents/:doc_id        Delete doc + purge Pinecone vectors

POST   /api/patients                 Create patient record

GET    /api/patients                 List patients (paginated, name fuzzy search)

GET    /api/patients/:id/timeline    NLP-extracted clinical events timeline

POST   /api/appointments             Create appointment

GET    /api/appointments             List appointments (date range filter)

GET    /api/sessions                 Session/query history

POST   /api/summarize                Summarize doc/note
                                     Body: { doc_id, mode: '1line'|'bullets'|'paragraph' }
                                     Response: { summary }

POST   /api/export/pdf               Generate PDF report
                                     Body: { doc_ids[], include_summary, include_charts }
                                     Response: { download_url }
```

---

5. FULL FRONTEND ARCHITECTURE & DETAILED GUI

5.1 Design System

```typescript
```

```
// theme/colors.ts
export const Colors = {
  // Primary clinical palette
  primary: '#1B6CA8',          // Deep medical blue
  primaryLight: '#E8F4FD',     // Soft blue background
  secondary: '#2E8B57',        // Forest green (positive indicators)
  accent: '#E8A838',           // Amber (alerts, highlights)

  // Surfaces
  background: '#F7F9FC',       // Off-white — reduces eye strain
  surface: '#FFFFFF',
  surfaceElevated: '#FFFFFF',
  border: '#E2E8F0',

  // Text hierarchy
  textPrimary: '#1A202C',      // Near-black
  textSecondary: '#4A5568',    // Gray
  textMuted: '#A0AEC0',

  // Status colors
  success: '#38A169',
  warning: '#DD6B20',
  error: '#E53E3E',
  info: '#3182CE',

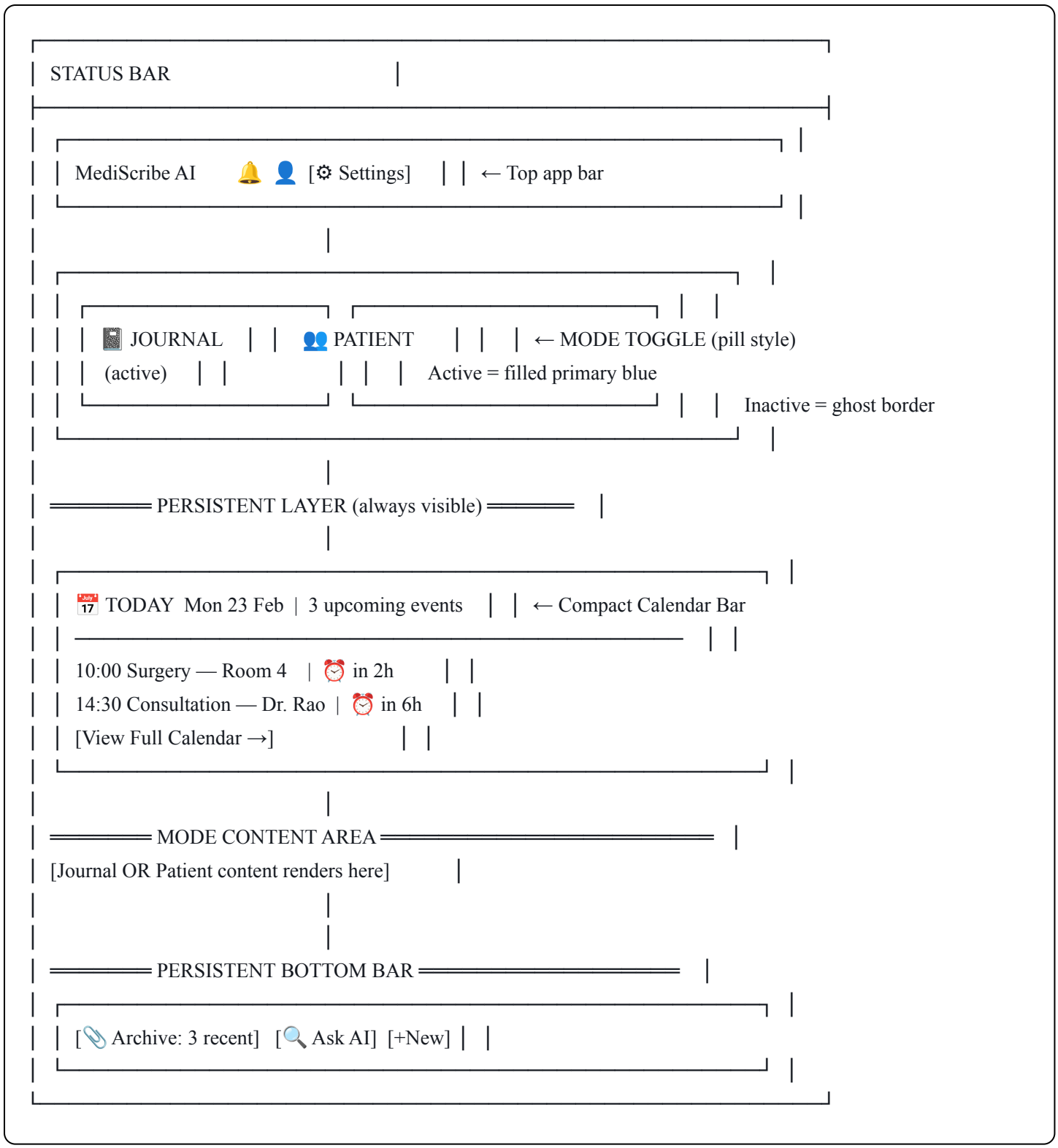
  // Journal mode accent
  journalAccent: '#7C3AED',    // Purple

  // Patient mode accent
  patientAccent: '#0891B2',    // Teal
};

// theme/typography.ts
export const Typography = {
  // iOS: SF Pro, Android: Roboto, fallback: System
  fontFamily: { regular: 'System', medium: 'System', bold: 'System' },
  sizes: { xs: 11, sm: 13, md: 15, lg: 17, xl: 22, xxl: 28, xxxl: 34 },
  lineHeights: { tight: 1.2, normal: 1.5, relaxed: 1.8 },
};
```

5.2 Screen-by-Screen GUI Specification

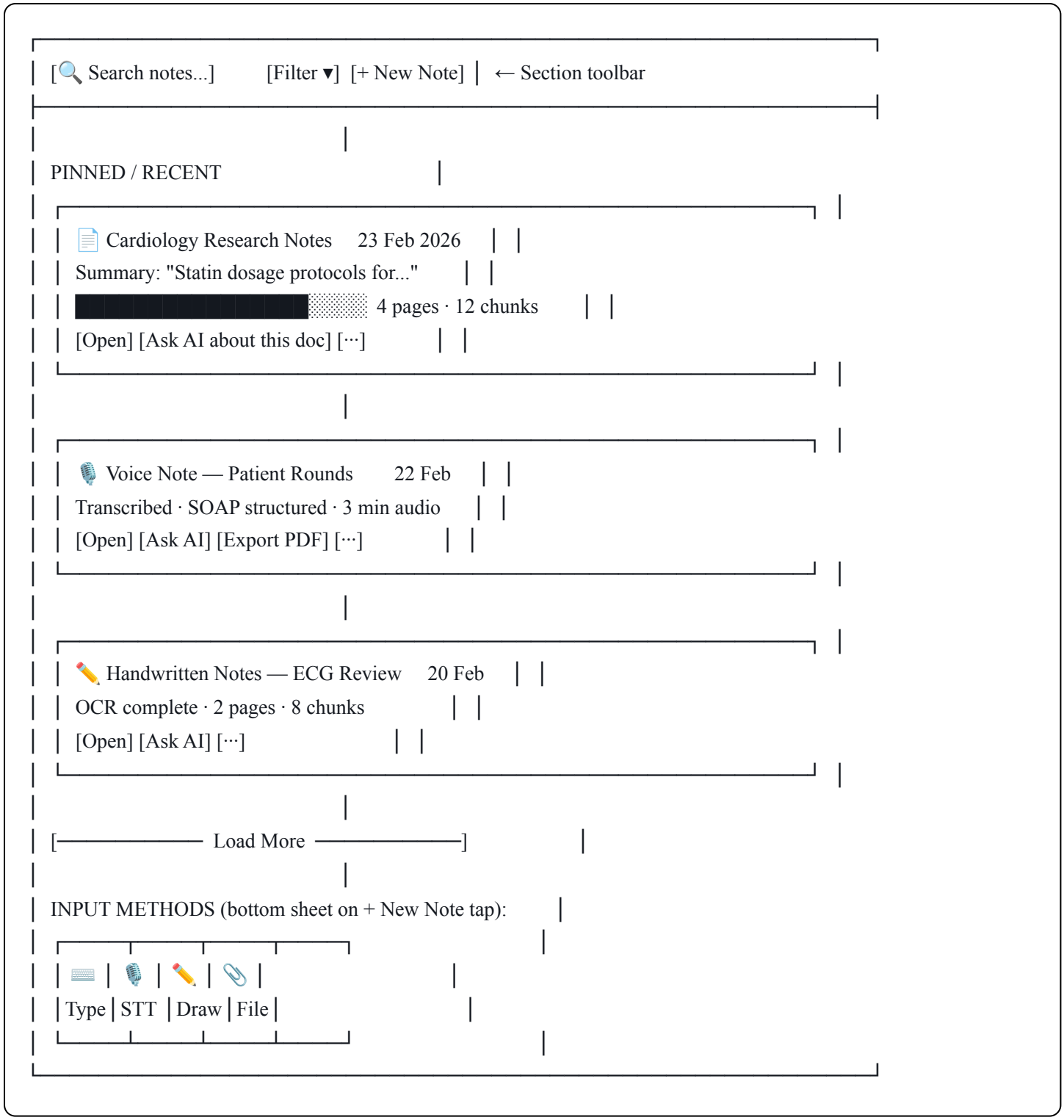
SCREEN 1: HomeScreen (Master Layout)



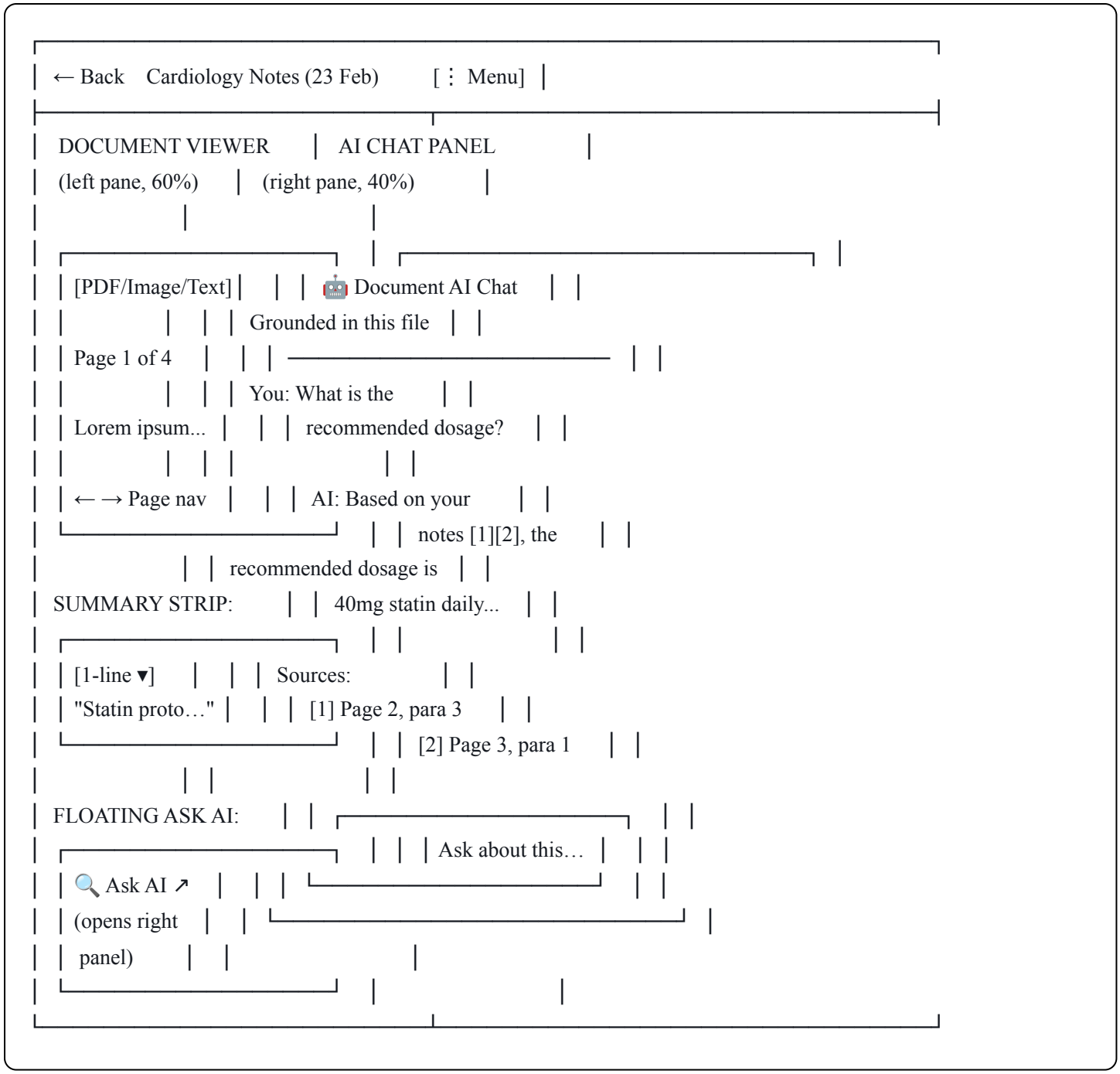
Animations:

- Toggle mode switch: smooth slide transition (300ms, ease-in-out)
- Calendar bar: collapses to single row when scrolling down
- Bottom bar: haptic feedback on "Ask AI" tap

SCREEN 2: Journal Mode Content



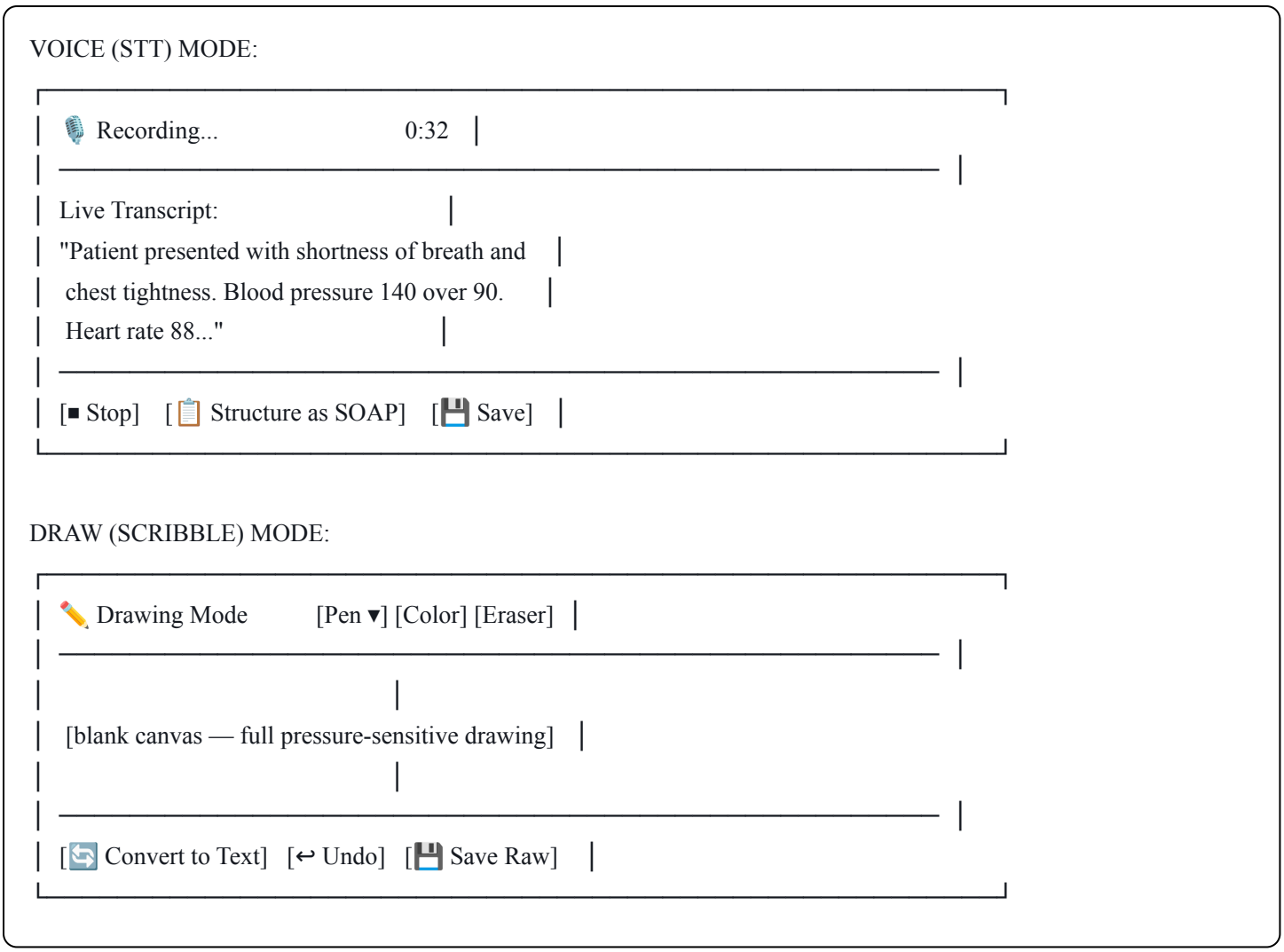
SCREEN 3: Document Detail + "Ask AI" Chat



On mobile (single pane): Document viewer is full screen. Floating "Ask AI" button (bottom-right FAB) slides up the chat panel as a bottom sheet at 70% height.

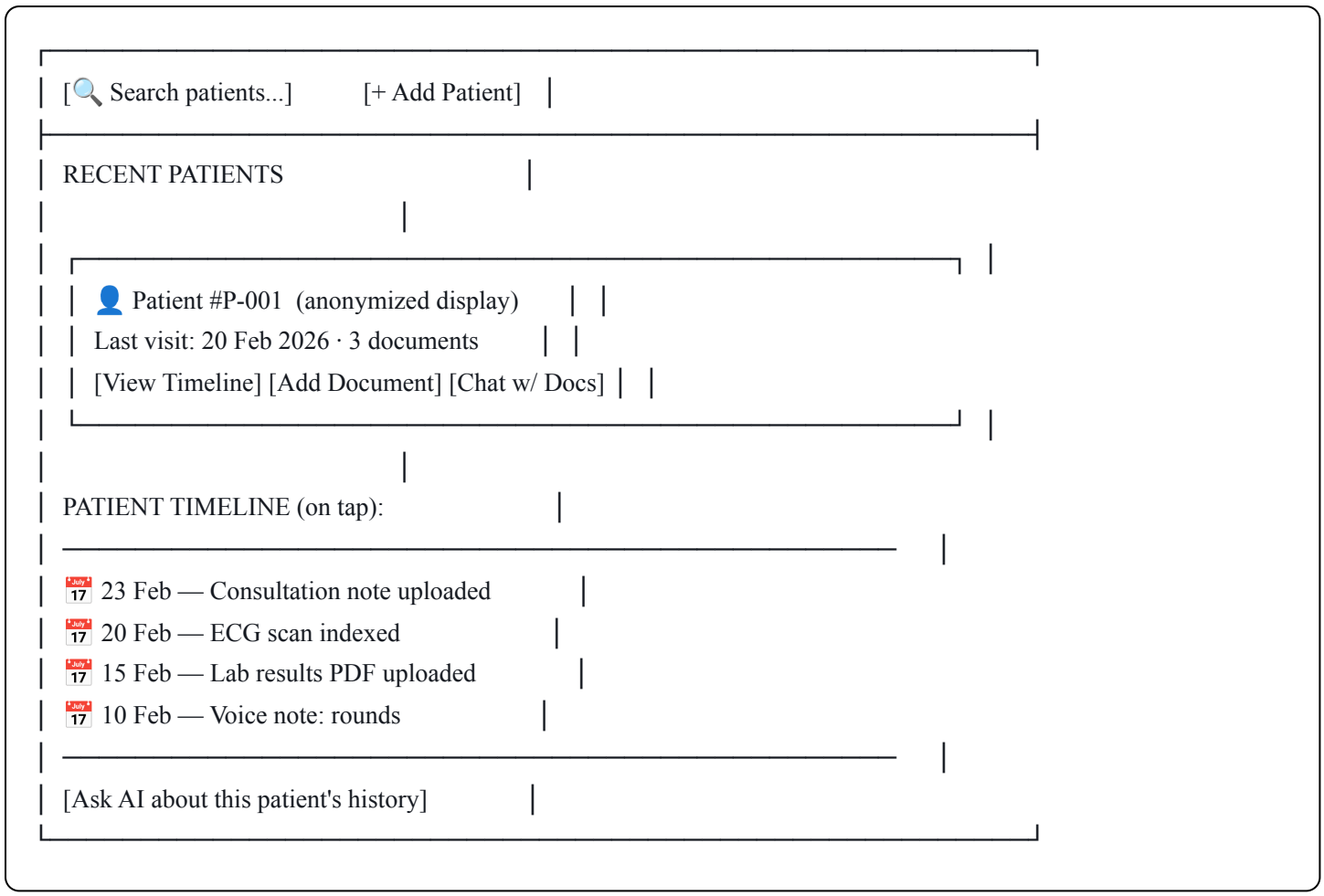
SCREEN 4: New Note — Input Mode Selector

VOICE (STT) MODE:

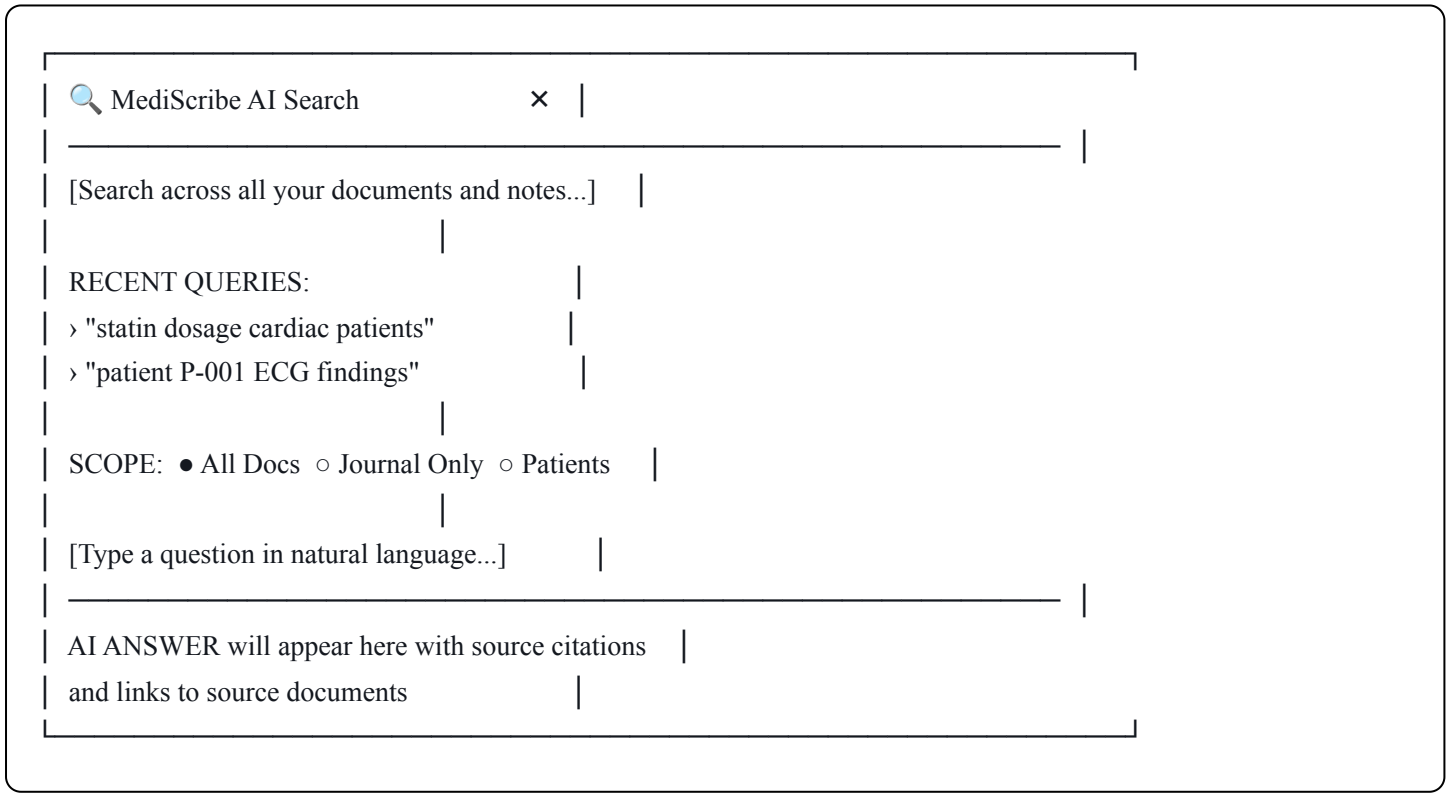


DRAW (SCRIBBLE) MODE:

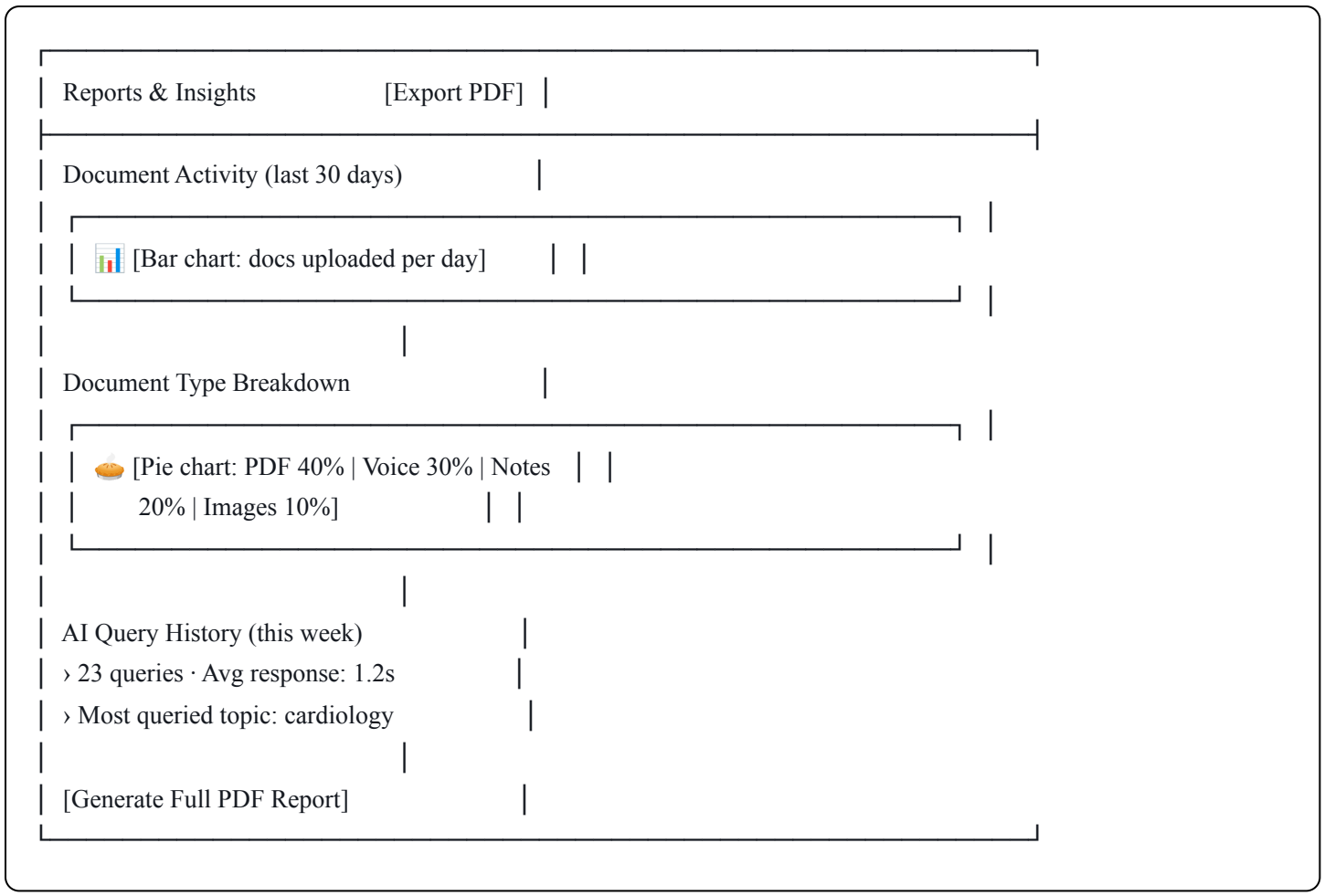
SCREEN 5: Patient Mode



SCREEN 6: AI Global Search (Persistent button)



SCREEN 7: Analytics / Reports



---

6. KEY COMPONENT IMPLEMENTATIONS

6.1 DocumentViewer with Floating "Ask AI" Button

typescript

```tsx
// components/journal/DocumentViewer.tsx
import React, { useState, useRef } from 'react';
import { View, TouchableOpacity, Text, StyleSheet, Animated } from 'react-native';
import { WebView } from 'react-native-webview';
import { useRAGQuery } from '../../hooks/useRAGQuery';
import ChatPanel from './ChatPanel';
import SummaryStrip from './SummaryStrip';

interface Props {
  docId: string;
  storageUrl: string;
  docTitle: string;
  docType: 'pdf' | 'image' | 'note';
}

export default function DocumentViewer({ docId, storageUrl, docTitle, docType }: Props) {
  const [chatOpen, setChatOpen] = useState(false);
  const [summaryMode, setSummaryMode] = useState<'1line' | 'bullets' | 'paragraph'>('1line');
  const chatAnimation = useRef(new Animated.Value(0)).current;
  const { query, response, isLoading } = useRAGQuery({ docScope: docId });

  const toggleChat = () => {
    setChatOpen(!chatOpen);
    Animated.spring(chatAnimation, {
      toValue: chatOpen ? 0 : 1,
      useNativeDriver: true,
    }).start();
  };

  return (
    <View style={styles.container}>
      {/* Summary strip */}
      <SummaryStrip docId={docId} mode={summaryMode} onModeChange={setSummaryMode} />

      {/* Document render */}
      <View style={styles.documentArea}>
        {docType === 'pdf' ? (
          <WebView source={{ uri: storageUrl }} style={styles.webview} />
        ) : (
          // Image or text render
          <View style={styles.textContent}>{/* render text/image */}</View>
        )}
      </View>

      {/* Floating "Ask AI" FAB */}
      {!chatOpen && (
```

```
        <TouchableOpacity style={styles.fab} onPress={toggleChat} activeOpacity={0.8}>
          <Text style={styles.fabText}>🔍 Ask AI</Text>
        </TouchableOpacity>
      )}

      {/* Chat Panel (bottom sheet on mobile) */}
      {chatOpen && (
        <ChatPanel
          docId={docId}
          docTitle={docTitle}
          onClose={() => setChatOpen(false)}
          onQuery={query}
          response={response}
          isLoading={isLoading}
        />
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F7F9FC' },
  documentArea: { flex: 1 },
  webview: { flex: 1 },
  textContent: { flex: 1, padding: 16 },
  fab: {
    position: 'absolute', bottom: 24, right: 24,
    backgroundColor: '#1B6CA8', borderRadius: 28,
    paddingHorizontal: 20, paddingVertical: 14,
    shadowColor: '#000', shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.3, shadowRadius: 8, elevation: 8,
  },
  fabText: { color: '#fff', fontWeight: '700', fontSize: 15 },
});
```

6.2 RAG Query Hook

typescript

```ts
// hooks/useRAGQuery.ts
import { useState, useCallback } from 'react';
import { getIdToken } from 'firebase/auth';
import { auth } from '../services/firebase';

interface RAGSource {
  doc_id: string;
  page?: number;
  text_preview: string;
  doc_title: string;
}

interface RAGResponse {
  answer: string;
  sources: RAGSource[];
  session_id: string;
}

interface UseRAGQueryOptions {
  docScope?: string;      // Restrict RAG to specific document
  patientScope?: string;  // Restrict RAG to patient's documents
}

export function useRAGQuery(options: UseRAGQueryOptions = {}) {
  const [response, setResponse] = useState<RAGResponse | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const query = useCallback(async (queryText: string) => {
    setIsLoading(true);
    setError(null);
    try {
      const token = await getIdToken(auth.currentUser!);
      const res = await fetch(`${process.env.EXPO_PUBLIC_API_URL}/api/query`, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Authorization': `Bearer ${token}`,
        },
        body: JSON.stringify({
          query: queryText,
          doc_scope: options.docScope,
          patient_scope: options.patientScope,
        }),
      });
      if (!res.ok) throw new Error('Query failed');
```

```
      const data: RAGResponse = await res.json();
      setResponse(data);
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Unknown error');
    } finally {
      setIsLoading(false);
    }
  }, [options.docScope, options.patientScope]);

  return { query, response, isLoading, error };
}
```

6.3 PDF Ingest Cloud Function

typescript

```ts
// functions/src/ingest/pdfIngest.ts
import * as functions from 'firebase-functions';
import * as admin from 'firebase-admin';
import { Storage } from '@google-cloud/storage';
import { ImageAnnotatorClient } from '@google-cloud/vision';
import { Pinecone } from '@pinecone-database/pinecone';
import OpenAI from 'openai';
import pdfParse from 'pdf-parse';

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });
const pinecone = new Pinecone({ apiKey: process.env.PINECONE_API_KEY! });
const vision = new ImageAnnotatorClient();
const db = admin.firestore();

interface ChunkMetadata {
  chunk_id: string;
  doc_id: string;
  owner_id: string;
  patient_id?: string;
  text_preview: string;
  page: number;
  offset: number;
  type: string;
  timestamp: number;
}

export const processPDF = async (docId: string, storagePath: string, ownerId: string, patientId?: string) => {
  const storage = new Storage();
  const bucket = storage.bucket(process.env.FIREBASE_STORAGE_BUCKET!);
  const file = bucket.file(storagePath);

  // 1. Download file
  const [buffer] = await file.download();

  // 2. Extract text
  let rawText = '';
  try {
    const data = await pdfParse(buffer);
    rawText = data.text;
  } catch {
    // Scanned PDF — fallback to Vision OCR
    const [result] = await vision.documentTextDetection({ image: { content: buffer } });
    rawText = result.fullTextAnnotation?.text || '';
  }

  if (!rawText.trim()) {
```

```
    await db.collection('documents').doc(docId).update({ status: 'error', error: 'No text extracted' });
    return;
  }

  // 3. Chunk
  const chunks = chunkText(rawText, 800, 150);

  // 4. Embed (batch of 100)
  const index = pinecone.index(process.env.PINECONE_INDEX!).namespace(`user_${ownerId}`);

  for (let i = 0; i < chunks.length; i += 100) {
    const batch = chunks.slice(i, i + 100);
    const embedResponse = await openai.embeddings.create({
      model: 'text-embedding-3-small',
      input: batch.map(c => c.text),
    });

    const vectors = batch.map((chunk, j) => {
      const metadata: ChunkMetadata = {
        chunk_id: `${docId}_chunk_${i + j}`,
        doc_id: docId,
        owner_id: ownerId,
        patient_id: patientId,
        text_preview: chunk.text.slice(0, 200),
        page: chunk.page,
        offset: i + j,
        type: 'pdf',
        timestamp: Date.now(),
      };
      return {
        id: metadata.chunk_id,
        values: embedResponse.data[j].embedding,
        metadata,
      };
    });

    await index.upsert(vectors);
  }

  // 5. Update Firestore
  await db.collection('documents').doc(docId).update({
    status: 'indexed',
    chunk_count: chunks.length,
    indexed_at: admin.firestore.FieldValue.serverTimestamp(),
  });
};
```

```
function chunkText(text: string, chunkSize: number, overlap: number) {
  const words = text.split(/\s+/);
  const chunks: { text: string; page: number }[] = [];
  let i = 0;
  while (i < words.length) {
    const chunk = words.slice(i, i + chunkSize).join(' ');
    chunks.push({ text: chunk, page: Math.floor(i / 500) + 1 });
    i += chunkSize - overlap;
  }
  return chunks;
}
```

6.4 Core RAG Query Function

typescript

```ts
// functions/src/rag/query.ts
import { Pinecone } from '@pinecone-database/pinecone';
import OpenAI from 'openai';
import Anthropic from '@anthropic-ai/sdk';

const pinecone = new Pinecone({ apiKey: process.env.PINECONE_API_KEY! });
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });
// → To swap to Anthropic Claude: const claude = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });
// → To swap to Google Vertex: import { VertexAI } from '@google-cloud/vertexai'

const SYSTEM_PROMPT = `You are MediScribe AI, a clinical documentation assistant.
Answer ONLY using the numbered sources provided below.
Always cite sources as [1], [2], etc. after each claim.
If the answer cannot be found in the sources, respond EXACTLY:
"Insufficient information in your documents."
Never add external knowledge or make assumptions beyond the sources.`;

export async function ragQuery(params: {
  query: string;
  ownerId: string;
  docScope?: string;
  patientScope?: string;
  topK?: number;
}) {
  const { query, ownerId, docScope, patientScope, topK = 5 } = params;

  // 1. Embed query
  const embedRes = await openai.embeddings.create({
    model: 'text-embedding-3-small',
    input: query,
  });
  const queryVector = embedRes.data[0].embedding;

  // 2. Build Pinecone filter
  const filter: Record<string, unknown> = { owner_id: { $eq: ownerId } };
  if (docScope) filter.doc_id = { $eq: docScope };
  if (patientScope) filter.patient_id = { $eq: patientScope };

  // 3. Dense vector search
  const index = pinecone.index(process.env.PINECONE_INDEX!).namespace(`user_${ownerId}`);
  const searchRes = await index.query({
    vector: queryVector,
    topK: topK * 3,  // over-fetch for reranking
    includeMetadata: true,
    filter,
  });
```

```
  // 4. Select top-K by score
  const contexts = (searchRes.matches || [])
    .sort((a, b) => (b.score || 0) - (a.score || 0))
    .slice(0, topK);

  if (contexts.length === 0) {
    return { answer: 'Insufficient information in your documents.', sources: [] };
  }

  // 5. Build prompt context
  const contextText = contexts.map((match, i) => {
    const m = match.metadata as Record<string, string>;
    return `[${i + 1}] (${m.doc_id}, page ${m.page || 'N/A'}): ${m.text_preview}`;
  }).join('\n\n');

  // 6. LLM generation
  const completion = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [
      { role: 'system', content: SYSTEM_PROMPT },
      { role: 'user', content: `SOURCES:\n${contextText}\n\nQUESTION: ${query}` },
    ],
    max_tokens: 800,
    temperature: 0.1,  // Low temp for factual clinical answers
  });

  const answer = completion.choices[0].message.content || 'Unable to generate answer.';

  // 7. Extract sources for frontend display
  const sources = contexts.map((match) => {
    const m = match.metadata as Record<string, string>;
    return {
      doc_id: m.doc_id,
      page: m.page,
      text_preview: m.text_preview,
      doc_title: m.doc_id,  // Resolve full title from Firestore in production
    };
  });

  return { answer, sources };
}
```

7. SECURITY & HIPAA COMPLIANCE IMPLEMENTATION

typescript

```typescript
// functions/src/api/authMiddleware.ts
import * as admin from 'firebase-admin';
import { Request, Response, NextFunction } from 'express';

export async function verifyFirebaseToken(req: Request, res: Response, next: NextFunction) {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }
  const token = authHeader.split('Bearer ')[1];
  try {
    const decodedToken = await admin.auth().verifyIdToken(token);
    // Enforce MFA for clinical data access
    if (!decodedToken.firebase?.sign_in_second_factor && req.path.includes('/patients')) {
      return res.status(403).json({ error: 'MFA required for patient data' });
    }
    (req as any).uid = decodedToken.uid;
    (req as any).email = decodedToken.email;
    next();
  } catch {
    return res.status(401).json({ error: 'Invalid token' });
  }
}

// Audit logger — call on every PHI access
export async function logPHIAccess(userId: string, action: string, resourceId: string) {
  await admin.firestore().collection('audit_logs').add({
    user_id: userId,
    action,
    resource_id: resourceId,
    timestamp: admin.firestore.FieldValue.serverTimestamp(),
    ip: 'logged-separately',  // Log from request in middleware
  });
}
```

Firestore Security Rules:

javascript

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users can only read/write their own data
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
    match /documents/{docId} {
      allow read, write: if request.auth != null &&
        resource.data.owner_id == request.auth.uid;
    }
    match /patients/{patientId} {
      allow read, write: if request.auth != null &&
        resource.data.owner_id == request.auth.uid;
    }
    match /audit_logs/{logId} {
      allow read: if request.auth.token.admin == true;
      allow write: if false;  // Only Cloud Functions write to audit logs
    }
  }
}
```

---

8. ENVIRONMENT VARIABLES REQUIRED

```
bash
```

```
# .env (never commit — store in GCP Secret Manager or Firebase Config)

# Firebase
FIREBASE_PROJECT_ID=mediscribe-ai-prod
FIREBASE_STORAGE_BUCKET=mediscribe-ai-prod.appspot.com

# OpenAI (primary LLM + embeddings)
OPENAI_API_KEY=sk-...

# Anthropic Claude (fallback LLM)
# Swap OPENAI_API_KEY references in llmRouter.ts with ANTHROPIC_API_KEY
ANTHROPIC_API_KEY=sk-ant-...

# Google AI Studio / Vertex (tertiary / summaries)
# Use for Gemini 2.0 Flash for cheap summarization
GOOGLE_AI_API_KEY=AIza...

# Pinecone
PINECONE_API_KEY=...
PINECONE_INDEX=mediscribe-docs
PINECONE_ENVIRONMENT=us-east-1-aws

# Google Cloud Vision (OCR)
GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json

# Whisper (STT) — can use OpenAI's Whisper API
WHISPER_API_KEY=${OPENAI_API_KEY}

# MyScript (Handwriting SDK)
MYSCRIPT_APP_KEY=...
MYSCRIPT_HMAC_KEY=...

# Mobile
EXPO_PUBLIC_API_URL=https://us-central1-mediscribe-ai-prod.cloudfunctions.net
```

---

9. PLATFORM-SPECIFIC NOTES

For OpenAI Playground / Codex:

Paste this entire spec. Request: "Generate the full TypeScript project scaffold with all files listed in Section 2, implementing the data models from Section 4.3, the API endpoints from Section 4.4, and the React Native components from Sections 6.1–6.4. Use environment variable stubs for all API keys."

For Claude (Anthropic):

Same as above. Add: "In `llmRouter.ts`, implement Claude as the primary option with: `const claude = new Anthropic(); const msg = await claude.messages.create({ model: 'claude-opus-4-6', max_tokens: 800, messages: [...] })`. Keep OpenAI as a swap-in via ENV flag `LLM_PROVIDER=openai|claude|gemini`."

For Google AI Studio:

Focus prompt on: "Implement the RAG query using Gemini File Search with built-in vector store. Replace Pinecone with `const genAI = new GoogleGenerativeAI(key); const model = genAI.getGenerativeModel({ model: 'gemini-2.0-flash' })`. Use Vertex AI Embeddings instead of OpenAI embeddings."

For Windsurf / Cursor (IDE):

Open the repo root, paste this prompt, and request: "Create all files from the directory structure in Section 2. Implement full backend in `/functions/src`, full mobile app in `/apps/mobile/src`. Run `npm install` and verify TypeScript compiles."

For Firebase Console Setup:

1. Create project: `mediscribe-ai-prod`
2. Enable: Authentication (Email + Phone MFA), Firestore, Storage, Cloud Functions
3. Set Firestore security rules from Section 7
4. Enable Google Cloud Vision API in GCP Console
5. Sign BAA with Google Cloud for HIPAA compliance

---

10. MVP SPRINT ROADMAP

Sprint 1 (2 weeks) — Core Plumbing: Firebase Auth → File upload to Storage → PDF ingest Cloud Function → Pinecone upsert → Basic Note Viewer UI → Mode toggle skeleton

Sprint 2 (3 weeks) — Intelligence Layer: RAG query endpoint → Document-scoped "Ask AI" button → Inline citation display → Voice recording + Whisper STT → SOAP note auto-structuring

Sprint 3 (4 weeks) — Full Feature Set: Scribble canvas + MyScript handwriting → Patient profiles + timeline → Appointment calendar with notifications → Analytics charts + PDF export → RBAC + audit logs → HIPAA compliance review

---

11. FINAL PITCH (use verbatim in hackathon presentation)

MediScribe AI unifies clinicians' scattered handwritten notes, scanned records, and voice dictations into a single, intelligent clinical workspace. Every AI-generated answer is grounded exclusively in the doctor's own trusted documents — never hallucinated, always cited. By combining an Obsidian-inspired knowledge graph with Scrivener-style research organization and a modern mobile-first journal UI, optimized specifically for

clinical workflows, MediScribe AI accelerates diagnosis, reduces cognitive overload, and delivers HIPAA-grade security — built for real doctors, deployed on Firebase, powered by RAG.

MediScribe AI — Build Specification v1.0 | Team: Adithya Baiju, MediScribe AI | Hackathon: Gravity-CIET 2026, MCET